SENIOR PROJECT EN1-2025

AI for Waste Sorting

Final Report

Submitted to

School of Information, Computer and Communication Technology
Sirindhorn International Institute of Technology
Thammasat University

May 2026

By
Kawisara Premwinai 6522770708
Palinee Sadibhakdi 6522770724
Jakrapat Wongpradu 6522771045

Advisor: Dr. Ekawit Nantajeewarawat

Acknowledgement

We express our sincere gratitude to Dr. Ekawit Nantajeewarawat, the project advisor, for providing us with invaluable advice and knowledge, enabling us to carry out this project with fairness and proficiency.

This project would not have been possible without the support and encouragement of both Dr. Ekawit Nantajeewarawat and the entire School of Information, Computer and Communication Technology, Sirindhorn International Institute of Technology at Thammasat University.

Abstract

This project aims to design and develop an AI waste classification system that aims to improve the accuracy and efficiency of traditional waste sorting. Using a YOLO object detection model, the system identifies and classifies waste types into recyclable, hazardous, and general wastes in real time by using a camera feed. Our project aims to support automated waste management processes by improving the process and reducing human error. Preliminary model results present the model's potential, with further improvements expected through model refinement and limitations on dataset expansion. Overall, this work contributes to more efficient waste-sorting solutions and supports sustainable environmental management practices.

Contents

Introduction

Environmental sustainability has become an urgent global priority, and effective waste management is essential in addressing this challenge. In many communities, waste sorting still depends on manual processes, which are slow, inconsistent, and vulnerable to human error. To enhance efficiency and accuracy, this project focuses on developing a real-time waste classification system using artificial intelligence and computer vision. The goal is to automatically detect and categorize waste into three main types: recyclable, hazardous, and general waste, through a live camera feed.

The project utilizes a YOLO-based deep learning model, a fast and widely adopted object detection framework known for its ability to classify and localize objects in a single processing step. The system is trained using supervised learning with labeled images, supported by techniques such as transfer learning and data augmentation to improve generalization in real-world conditions. This enables the model to recognize diverse waste objects despite variations in lighting, orientation, and background environments.

1. Project Concept

1.1 Summary

The goal of this project is to use computer vision and real-time object detection to design and develop an AI-powered waste sorting and sorting system. From photos taken by integrated cameras, the system uses YOLO-based models to identify common waste types like batteries, paper, plastic bottles, and mobile phone. Once detected, the system automatically directs the waste item to the correct bin. The goal is to reduce manual labor, improve sorting accuracy, and support more sustainable waste management practices. At the current stage, the basic YOLO model has been trained and tested on a custom dataset, which serves as the foundation for improving accuracy, robustness, and real-time detection.

1.2 Motivation

The motivation behind this project stems from the growing need for efficient waste management solutions in modern communities. It was observed that there is still confusion or misunderstanding regarding waste types in waste separation, and manual sorting is time-consuming and may be inaccurate, exposing workers to hazardous materials. Additionally, incorrect sorting leads to the contamination of recyclable materials, further reducing the efficiency of recycling programs as well as limiting waste by type. By utilizing an AI-powered detection system, this project aims to reduce human error, promote safer and more accurate waste management, and support the development of smart and sustainable infrastructure today.

1.3 User and Benefit

The system is intended for use by educational institutions such as the SIIT faculty, other faculties within the university, and shopping malls.

Users benefit

- More accurate waste disposal processes without needing to understand complex recycling rules.
- Facility managers can reduce labor costs, improve cleanliness, and enhance recycling rates.

Overall, both individual users and organizations benefit from improved convenience, safety, and sustainability.

1.4 Typical Usage

A typical usage Inserting a waste item into the system's input tray initiates a typical usage scenario. The camera captures an image, the YOLO model analyses it, and the system identifies the waste category within seconds. Based on the predicted class, the sorting mechanism directs the item into its category. There is very little user interaction because the process is completely automated. Malls, department stores, academic staff, and smart trash cans made for areas with sparse populations can all benefit from this workflow.

1.5 Main Challenges

- **Dataset limitations:** The model may struggle if the training dataset is small, unbalanced, or lacks variation in waste shapes, colors, or lighting conditions.
- **Class similarity issues:** Some waste categories look visually similar, causing misclassification unless the model is trained with very clear distinctions.

2. Requirements Specification

2.1 System Description

The AI Waste Sorting System is an intelligent automation platform capable of automatically detecting, categorizing, and sorting waste through the object detection model based on YOLO. If the user places an item in the input tray, the camera will capture that image and send it into the YOLO model for object identification. The object will be assigned to a class label, such as glass, battery, plastic bottle, and light bulb. Unlike other traditional image-classification models, YOLO performs real-time object detection, enabling the system to locate the item within the frame and classify it with high accuracy. According to the identified category, the system will automatically move the waste into the respective sorting compartment. This system improves the efficiency of waste management by saving much manual labor, minimizing human error, and developing smart eco-friendly infrastructure suitable for educational, commercial, and public environments.

2.1.1 Perspective

The relationship between the Al waste sorting system and users is shown in Figure 1.

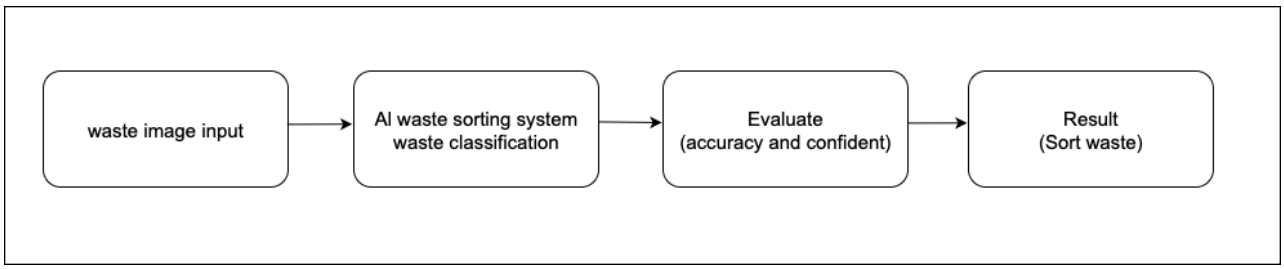


Figure 1

When the user places the discarded items on the tray, it will then go into the AI waste sorting system. The camera acquires an image. Decision logic validates confidence, then selects the target bin.

2.1.2 Functions

The functions of the system are shown in Figure 2.

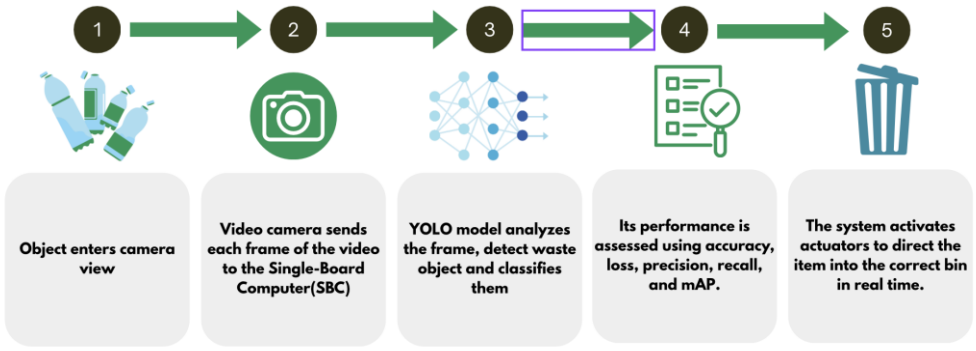


Figure 2

The functions are summarized as follows:

- **Input (Camera):**
  The system begins with capturing real-time images of waste objects through a connected webcam. The user aims for the camera toward the waste item, and each frame of the video is sent to the computer (running Google Colab) for processing. This step enables continuous real-time data input for classification.

- **Model:**
  The AI model, built using a pretrained YOLO architecture, processes the captured image by performing real-time object detection to identify the waste category. The model has been fine-tuned on a labeled dataset containing various waste types such as plastic, paper, and battery. During execution, YOLO analyzes spatial and visual features within the image, locates the object, and outputs the predicted class along with a confidence score.

- **Evaluation:**
  The system evaluates the model's performance using standard machine learning metrics, including accuracy, loss, precision, and recall. These metrics help measure how well the model distinguishes between waste categories and identify potential areas for improvement, such as data imbalance or model overfitting.

- **Output (Automatic Sorting System):**
  After the prediction is made, the system directly controls hardware components such as motors or actuators. These components guide the detected waste item into the correct bin automatically in real time.

2.2 Requirements

2.2.1 General Requirements (GR)

GR1 System shall classify waste into multiple categories.
GR2 System shall be easily deployable on a computer or embedded device.
GR3 System shall allow future model updates or retraining.

2.2.2 Dataset (D)

D1 System shall split data into training, validation, and test sets.
D2 Each class must contain at least 150 images.
D3 Images shall be resized to 160 × 160 pixels and normalized before training.

2.2.3 Model Development (MD)

MD1 Model training process shall stop early if validation accuracy does not improve.
MD2 System shall report overall accuracy percentage after training.
MD3 System shall evaluate model accuracy, precision, recall, and F1-score.

2.3 Mock-Up

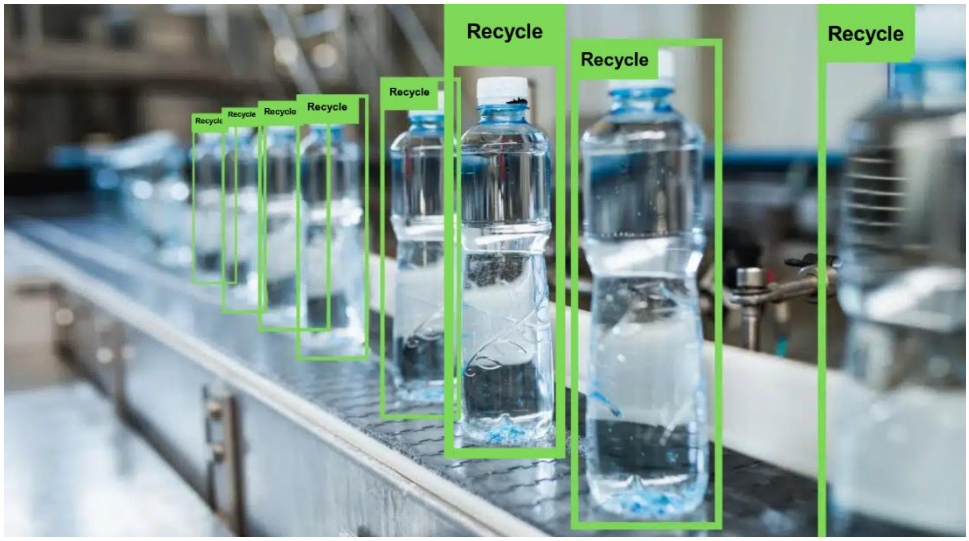


Figure 3

In this mockup, a series of plastic bottles are placed on a conveyor belt, representing waste items moving through a sorting environment. The AI model processes each frame captured by the camera and detects the bottles as recyclable objects. The green bounding boxes indicate that the system has successfully recognized and classified the items as belonging to the *"Recycle"* category.

This visualization demonstrates the expected behavior of the system during deployment, real-time object detection, and classification of waste materials using the AI model. The bounding boxes and labels serve as immediate feedback to confirm accurate identification. Although the current implementation operates on a computer through Google Colab.

3. Design Specification

3.1 System Architecture

The AI Waste Sorting System consists of four major components: Camera Input, Processing Module, Classification Output, and the User Interface. Each component works together to enable real-time detection and classification of waste items using a YOLOv11-based object detection model.

The system begins with the Camera Input module, where a webcam or external camera continuously captures real-time images or video frames of the waste item placed in front of it. This component is responsible for:

- Providing continuous visual data for processing
- Ensuring each frame is transmitted to the processing pipeline
- Allowing the system to operate in real time as the user positions waste in view of the camera

The captured frames are then forwarded to the Processing Module, which is the core of the system, and each subcomponent contributes to transforming raw camera data into a meaningful prediction.

1. Pre-processing
2. Object Detection (YOLOv11)
3. Waste Classification (Map Class)
4. Hardware specification

The User Interface (UI) serves as the final point of interaction. It receives the predicted class from the processing module and displays the results in real time.

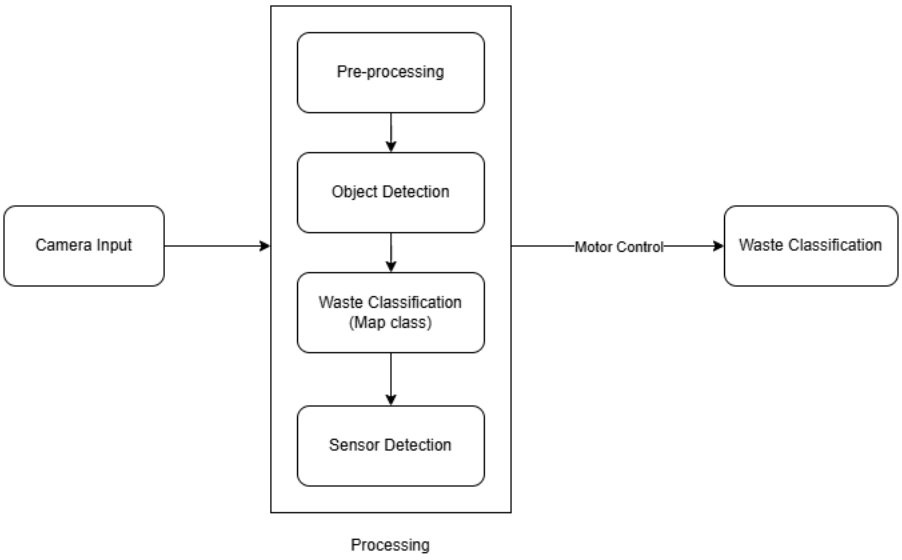


Figure 4

3.2 Detailed Design

The system proposed for waste type screening is designed to detect and categorize waste, offering an efficient and accurate solution that leverages the latest advancements in AI and data processing.

3.2.1 Input (Camera)

The input subsystem is responsible for the capture of high-quality images of the waste item placed on the tray of the system. For this, a digital camera or webcam mounted in a fixed position above the tray captures the object on the tray so that it is completely visible and properly framed whenever its image is captured. Upon placing an item on the tray, detection by a sensor triggers the camera after a short delay to allow the object to stabilize, which captures the picture. This captured image is then resized, normalized, and transformed into the correct input tensor format that is needed as per the requirements of the model YOLO. This preprocessing will ensure that the system gets clean, uniform input for reliable detection and classification.

3.2.2 Pre-processing Module

The input from the camera provides raw video frames that must be processed before sending them to the YOLO object detection model. The primary purpose is to prepare each frame in a format suitable for efficient and accurate inference. When the system captures each frame, it is first converted into an RGB image array representing the pixel values of the current video frame. This image will be pre-processed by resizing it to 640 × 640 pixels, which is the required input resolution for the YOLO model used in this project. After resizing, the frame is normalized by dividing all pixel values by 255, so that the data falls within the standard range (0,1) expected by the model. Once these pre-processing steps are completed, the processed frame is sent to the YOLO inference module for waste detection and classification.

3.2.3 Detection Module

The purpose of the Detection Module is to analyze the camera frames and identify waste objects in real time, producing outputs that the system can use for classification and sorting. This module consists primarily of the YOLO object detection model, which performs both localization and classification of waste items within the camera's view.

First, the YOLO model processes the pre-processed frame and detects objects present in the scene. It identifies each waste item and assigns it to a predefined class: recyclable, hazardous, or general waste, while also generating a bounding box that defines the object's 2D coordinates within the frame. The model outputs confidence scores indicating the reliability of each prediction.

All outputs from this module and bounding boxes, predicted classes, and confidence levels will be passed to the display, where they are visualized for the user in real time. These outputs also serve as input to an automated sorting mechanism, enabling the system to physically route waste items to their appropriate bins.

3.2.3.1 Object Detection Model, YOLOv11

YOLO (You Only Look Once) models are real-time object detection systems that identify and classify objects in a single pass of the image. In other words, the model only

looks at the image once and, from this ‘single pass’, is able to identify objects in the image. This is different from previous models that required multiple passes to process an image. Since this process happens in real-time, it works at an incredible speed.

YOLOv11 employs an anchor-free detection mechanism. The model directly predicts object centers, bounding box dimensions, and class probabilities at each location in the feature map. This simplifies training and improves detection accuracy for objects of various shapes and sizes, which is an important advantage for real-world waste items.

Each input frame is resized to 640 x 640 pixels, which is the standard input resolution for YOLOv11. The model then extracts the features using a deep convolutional backbone with a multi-scale feature fusion network that improves the detection performance for all sizes of objects (small, medium, and large). The detection outputs three key components for every detected waste item:

- Box coordinates (x, y, width, height)
- Confidence score shows the accuracy of the predicted object
- Class probabilities for the waste categories

These outputs are refined using Non-Maximum Suppression (NMS) or the model’s built-in filtering process to remove overlapping or low-confidence detections. The final predictions are then directly utilized by the integrated system for real-time visualization and automated sorting operations. Based on the detected object class and confidence score, the system immediately triggers the corresponding hardware response, enabling accurate and efficient waste classification and physical sorting in real time.

3.2.4 Hardware specification

The proposed waste classification system consists of four main hardware components: a camera module, sensor module, single-board computer (SBC), and a motor system.

The camera module captures real-time images of waste items and sends them to the NVIDIA Jetson Nano for processing. The Jetson Nano runs the YOLOv11 object detection model to classify recyclable and hazardous waste.

A sensor module is used to detect the presence of waste and support the classification of general waste when no specific class is detected by the model.

The NVIDIA Jetson Nano functions as the central processing unit of the system. It performs image processing, runs the deep learning model, processes sensor inputs, and controls the sorting mechanism.

The motor system (e.g., servo) is responsible for rotating the tray or directing waste into the appropriate bin based on the classification result.

Together, these components enable real-time detection and automated waste sorting.

1) Dataset

Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
images	File folder					12/1/2025 6:09 AM
labels	File folder					12/1/2025 6:09 AM
classes	Text Document	1 KB	No	1 KB	15%	12/1/2025 6:09 AM
notes	JSON Source File	1 KB	No	1 KB	68%	12/1/2025 6:09 AM

Figure 5

For this project, a custom dataset was developed specifically for real-time waste classification. We initially selected relevant datasets using Google Dataset Search, including sources such as Kaggle and Roboflow, to ensure that the data aligned with our project requirements. These datasets were chosen because they contain images of common waste items found in everyday environments such as shopping malls, retail stores, and small buildings. This selection allows the model to learn and recognize realistic waste objects and classify them accurately.

In addition to publicly available datasets, we further improved the dataset by capturing our own images under real-world conditions similar to the system's deployment environment. This enhancement helps increase the model's robustness and accuracy by adapting it to actual lighting conditions, backgrounds, and object variations.

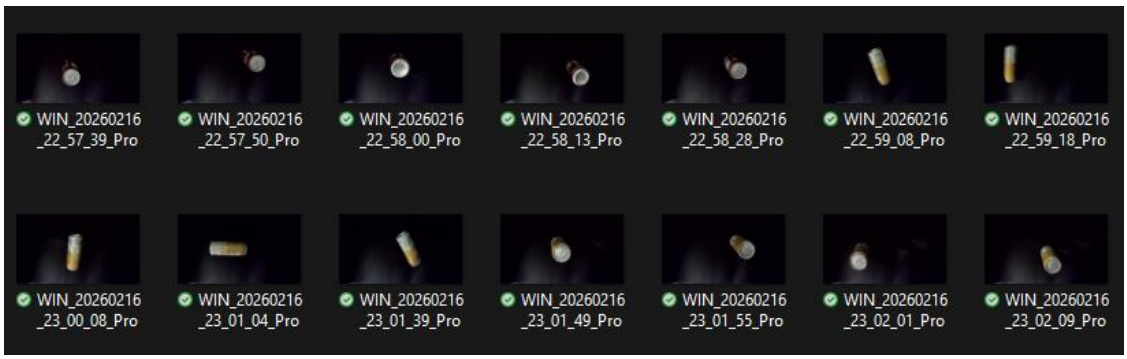


Figure 6

The dataset contains 1,166 labeled images. All images were manually labeled by our team to ensure accuracy and consistency. The dataset includes ten waste classes: Battery, Can, Cutlery, Lightbulb, Face mask, Smartphone, Water bottle, Milk Carton, Plastic Cup, and Plastic Straw. Each class contains at least 100 images. These classes were selected to represent a mixture of recyclable and hazardous categories. The dataset is organized into the standard YOLO structure, consisting of:

- An images folder
- A labels folder with .txt files for all images
- A class list in a .txt file contains nine categories

All images were resized during preprocessing to ensure compatibility with YOLOv11's 640×640 input requirement. We have contributed to the model's ability to detect and classify waste items accurately in real time.

2) Performance Evaluation

We have set the criteria for training by using the same amount and quality of datasets to ensure a fair comparison. The purpose of this evaluation is to assess the performance differences between a traditional image classification model (MobileNetV2) and a modern object detection model (YOLOv11) when applied to the waste-classification task.

1. Performance: It measures how accurately the model can identify or detect objects, using metrics like accuracy, precision, recall, and mAP.
2. Inference Efficiency: How quickly and smoothly the model can make predictions, representing speed per frame, how well it runs in real time, and how much computing power it needs.
3. Task Suitability: How appropriate the model is for the specific job. For example, detecting multiple objects, showing their positions, or real-time camera input.

	MobileNetV2	YOLOv11
Performance	- 84% validation accuracy (classification only)	- Precision/Recall: 0.84–0.89 - mAP50 ~0.88–0.91 (detection + classification)
Inference Efficiency	- very fast per image - single-object classification	- slightly slower - multiple-object classification
Task Suitability	- only classifies one object per image - no bounding boxes	- detects & classifies multiple waste items with localization

3.2.5 Evaluation

The evaluation subsystem focuses on measuring and ensuring the performance, reliability, and safety of the waste sorting model. A separate test dataset, containing diverse images with varying lighting conditions and item orientations, is used to evaluate the model's precision and recall. These metrics help measure how accurately the system identifies each type of waste and how often it makes mistakes or miss detections. Confidence scores also play a critical role in system decision-making. By tuning the confidence threshold, the system can adjust its balance between precision and recall. Higher thresholds improve reliability but may reject more items.

3.3 Test Result

After training the YOLOv11 model, the system was tested on a set of images containing various waste objects. The model successfully detected and classified multiple categories, including cans, light bulbs, batteries, milk cartons, plastic bottles, and cutlery. For each detected object, the model generated bounding boxes along with confidence scores, demonstrating its ability to localize and recognize items accurately.

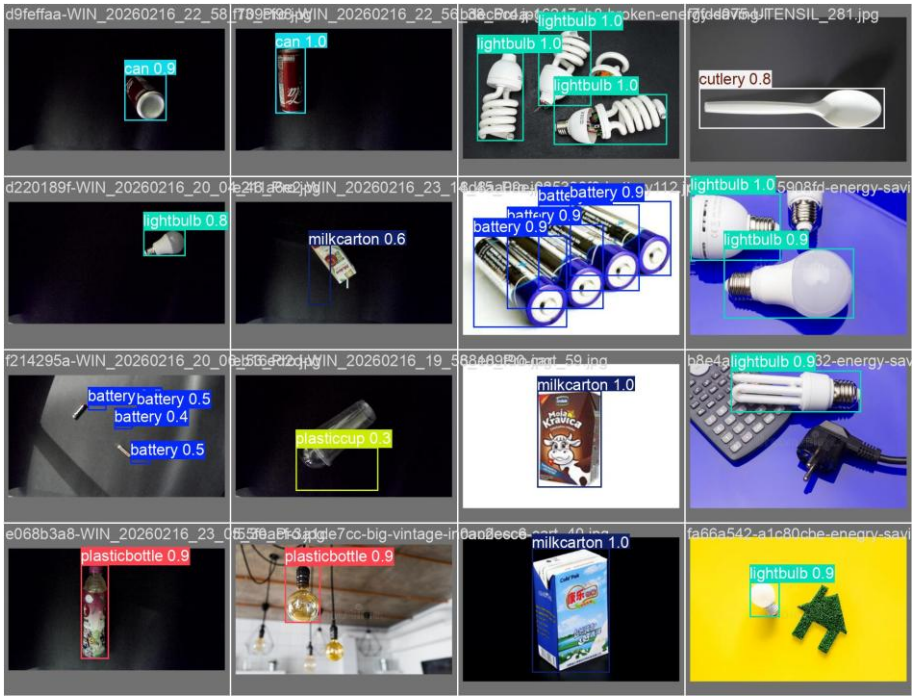
The YOLOv11 model was trained and then its performance was assessed using F1-score, precision, recall, and mean Average Precision (mAP). The model produced excellent overall performance with an F1-score of 0.91 at a confidence level of 0.709, indicating that there was a good balance between precision and recall.

The Precision-Confidence graph demonstrates that prediction accuracy increases steadily with confidence level, ultimately reaching 1.00 precision value. This means that predictions that are substantially confident will almost always be accurate.

In addition, the Precision-Recall graph shows a combined mean Average Precision (mAP) of 0.927 at 0.5 confidence. These two graph representations indicate that the additional categories are consistent in terms of reliability of the model.

The Recall-Confidence graph additionally provides a representation of the predictable trade-off between recall and confidence levels. The higher the confidence level decreases, the greater the increment of recall decreases as the model selects fewer candidates. This result shows that the YOLOv11 model is consistent and has been optimized to reliably detect multiple categories of objects.

Overall, the evaluation demonstrates that the YOLOv11 model is accurate and reliable in terms of performance and can be used for real-time waste classification and for effective integration into an automated sorting system.



Figures 7

3.3.1 Performance Evaluation

The YOLO-based waste sorting model was evaluated using four key performances metrics to assess its classification accuracy and reliability.

1) F1-Confidence Curve Analysis

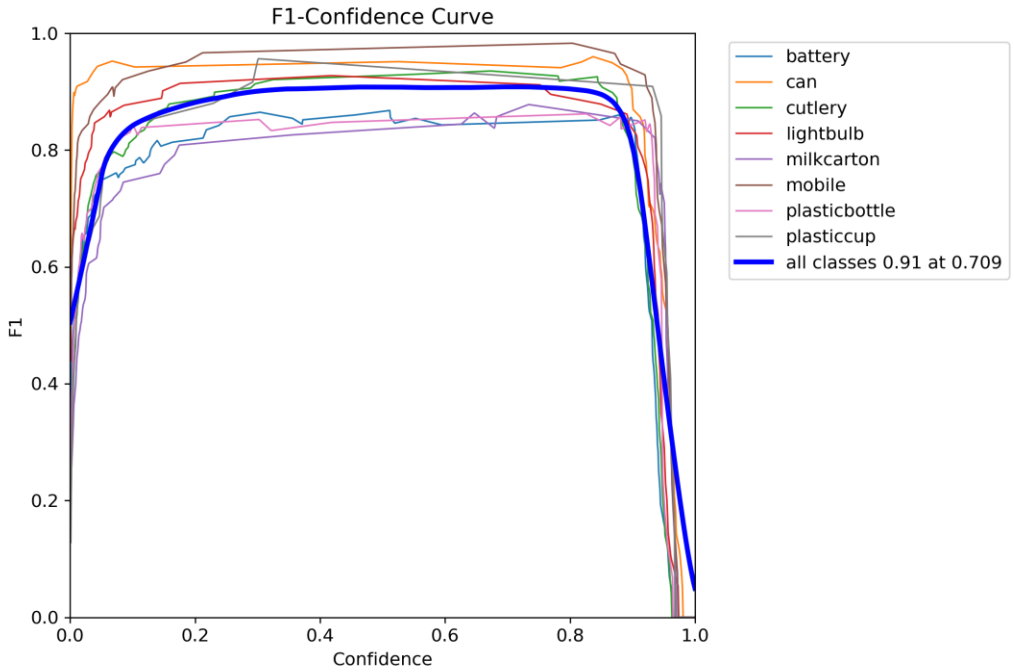


Figure 8

The F1-Confidence curve shows how well the model balances finding objects (recall) and being accurate about them (precision). The model scores 0.91 at a confidence level of 0.709, which means excellent balance at both detecting waste items and correctly identifying what they are.

Looking at individual categories, battery, can, and mask perform particularly well, with scores consistently above 0.90. This means the system rarely makes mistakes with these items. On the other hand, cardboard and mobile phones show more inconsistent results, which suggests that the model sometimes struggles to identify these items correctly. This could be because these objects look similar to other waste types, or the model simply needs more examples to learn from. The drop-off in performance at very high confidence levels (above 0.90) is normal and happens because the model makes fewer predictions when it's extremely selective.

2) Precision-Confidence Curve Analysis

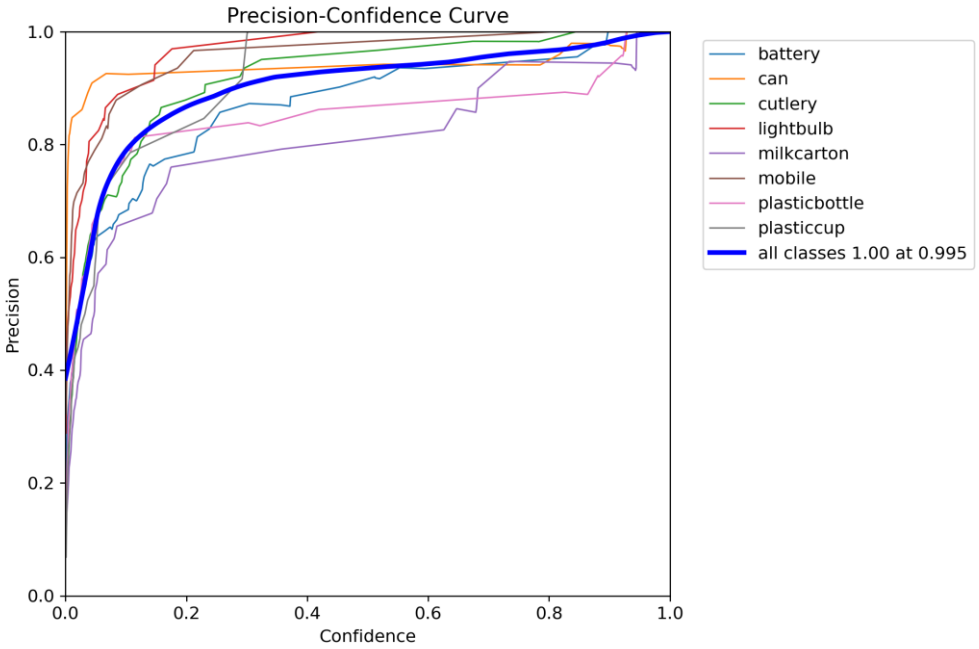


Figure 9

The Precision-Confidence curve measures the model's accuracy when making predictions at different confidence thresholds. The system achieves perfect precision (1.00) at a confidence level of 1.00, meaning that when the model is highly confident about its predictions, it is virtually always correct.

This upward trend across all classes indicates that higher confidence predictions are more reliable, which is valuable for setting operational thresholds in the waste sorting system.

3) Precision-Recall Curve Analysis

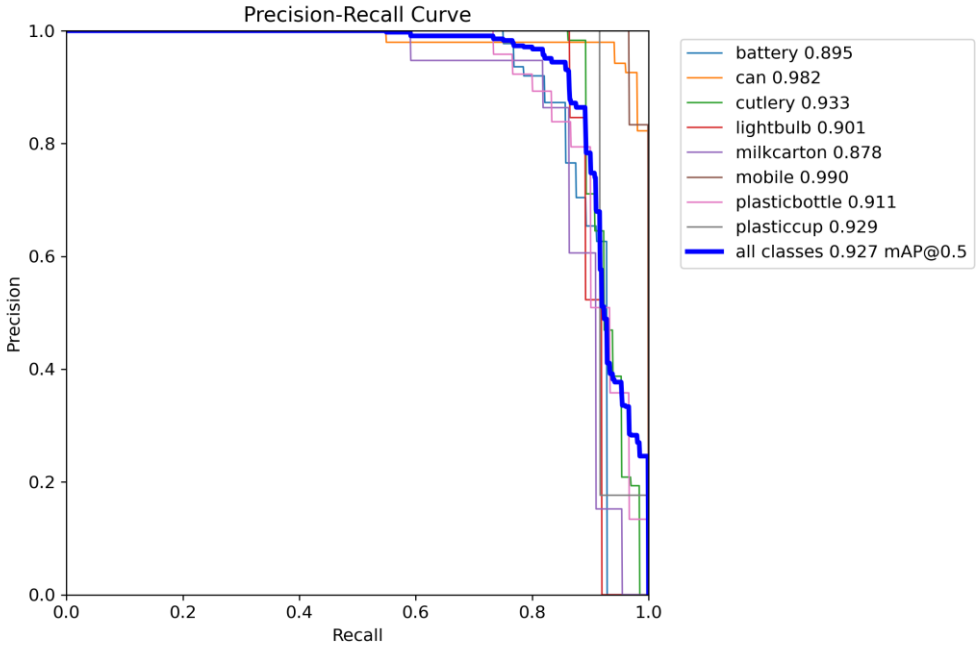


Figure 10

The Precision-Recall curve evaluates the trade-off between precision and recall independent of confidence threshold, providing insight into the model's overall detection capability. The system achieves an overall mean Average Precision (mAP) of 0.927 at an Intersection over Union (IoU) threshold of 0.5, which represents excellent detection performance.

Class-specific mAP scores reveal strong performance across most categories:

- Exceptional Performers (mAP ≥ 0.95): Mobile (0.990), Can (0.982)
- Strong Performers (0.92 – 0.95): Cutlery (0.933), Plastic Cup (0.929)
- Moderate Performer: Battery (0.895), Lightbulb (0.901), Milk Carton (0.878), Plastic Bottle (0.911)

The lower performance in cardboard and mobile phone categories suggests these objects may share visual features with other classes or require additional training examples to improve detection accuracy.

4) Recall-Confidence Curve Analysis

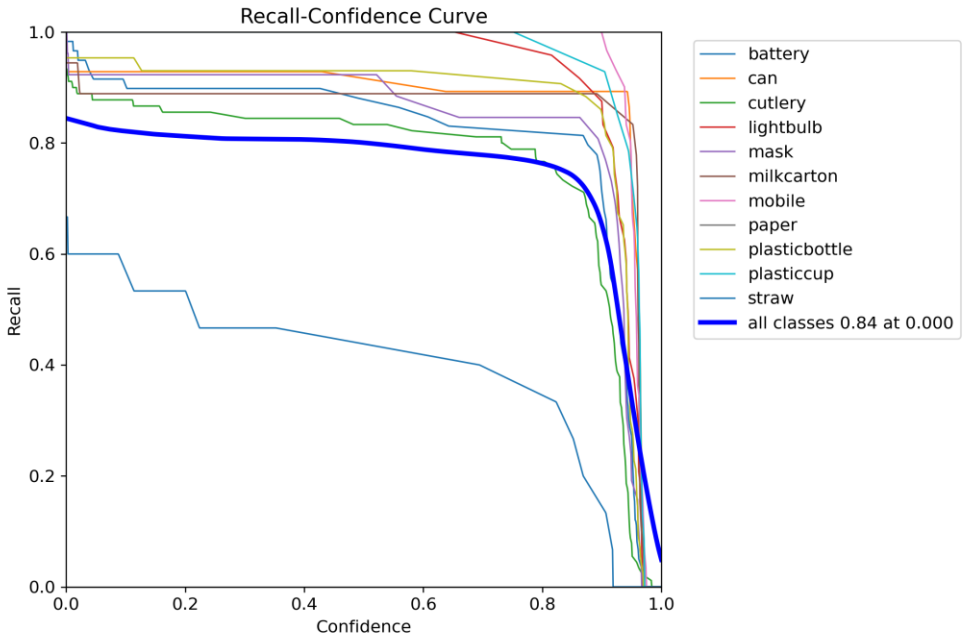


Figure 11

The Recall-Confidence curve demonstrates the percentage of actual objects successfully detected at each confidence level. The model achieves a recall of 0.84 at minimum confidence (0.000), indicating that the system can detect 94% of all waste items present in the input stream.

All classes maintain high recall rates until the confidence threshold exceeds approximately 0.80. This plateau at lower confidence levels is desirable for a waste sorting application, as it ensures consistent object detection across varying conditions and object presentations.

3.4 Training Process

To prepare the dataset for training the YOLOv11 model, the compressed data file (data.zip) was first uploaded to the Google Colab environment. The dataset was then extracted using the standard unzip command in Colab. It extracts the contents of *data.zip* into a new directory. Organizing the dataset in this directory allows the model to access images and label files in a structured format required for training. After extraction, the dataset becomes ready for preprocessing and model training.

Figure 12

A Python script was implemented to automatically organize the images and label files into the appropriate directory structure required by YOLOv11. The script begins by defining the source directory containing the raw dataset and the target directory where the final structure will be created. It first checks whether the images and labels folders exist, ensuring that both the image files and their corresponding annotation files are available. The program then creates the necessary subdirectories inside the target folder.

All image files in the dataset are collected and randomly shuffled to prevent ordering bias. Using a predefined train–validation split ratio (90% training and 10% validation), the script calculates the number of files allocated to each set. After determining the split, a helper function copies each image and its corresponding label file into the appropriate destination folder. If a label file is missing, the script notifies the user but still proceeds with copying the image. Additionally, important metadata files such as classes.txt and note.json are also transferred to the target directory to maintain dataset consistency. This automated splitting process ensures that the dataset is cleanly organized, prevents human error, and prepares the data in a format fully compatible with the YOLOv11 training

```python
import os
import random
import shutil
from pathlib import Path

SOURCE_DIR = "/content/custom_data"
TARGET_DIR = "/content/data"
TRAIN_RATIO = 0.9

images_dir = os.path.join(SOURCE_DIR, "images")
labels_dir = os.path.join(SOURCE_DIR, "labels")

if not os.path.exists(images_dir):
    raise ValueError("images folder not found at /content/custom_data/images")
if not os.path.exists(labels_dir):
    raise ValueError("labels folder not found at /content/custom_data/labels")

train_img = os.path.join(TARGET_DIR, "train/images")
train_lbl = os.path.join(TARGET_DIR, "train/labels")
val_img = os.path.join(TARGET_DIR, "validation/images")
val_lbl = os.path.join(TARGET_DIR, "validation/labels")

for d in [train_img, train_lbl, val_img, val_lbl]:
    os.makedirs(d, exist_ok=True)

print("Created folder structure under:", TARGET_DIR)

image_files = sorted([p for p in Path(images_dir).glob("*") if p.suffix.lower() in [".jpg", ".png", ".jpeg"]])
total_images = len(image_files)

print(f"Found {total_images} images.")

random.shuffle(image_files)

split_index = int(total_images * TRAIN_RATIO)

train_files = image_files[:split_index]
val_files = image_files[split_index:]
```

```python
print(f"Train images : {len(train_files)}")
print(f"Val images   : {len(val_files)}")

def copy_pair(image_path, dest_img_folder, dest_label_folder):
    img_name = image_path.name
    base = image_path.stem
    label_file = base + ".txt"
    label_path = os.path.join(labels_dir, label_file)

    shutil.copy(image_path, os.path.join(dest_img_folder, img_name))

    if os.path.exists(label_path):
        shutil.copy(label_path, os.path.join(dest_label_folder, label_file))
    else:
        print(f"⚠ No label found for {img_name}, skipping label.")

for img in train_files:
    copy_pair(img, train_img, train_lbl)

for img in val_files:
    copy_pair(img, val_img, val_lbl)

extra_files = ["classes.txt", "note.json"]
for ef in extra_files:
    src = os.path.join(SOURCE_DIR, ef)
    if os.path.exists(src):
        shutil.copy(src, os.path.join(TARGET_DIR, ef))
```

Figures 13 & 14

To streamline the training setup, a Python function was implemented to automatically generate the data configuration file required by YOLOv11. This function eliminates the need to manually write the YAML file and reduces errors related to incorrect paths or class definitions. The script begins by reading a text file, which contains the list of waste categories used in the project. The function loads each class name, counts the total number of classes, and stores them in a list.

Next, it constructs a data dictionary that includes the dataset root path, the locations of the training and validation of image folders, the number of classes, and the corresponding class names. Once the dictionary is complete, the function writes the content into a new dataset file using the PyYAML library. This automated approach ensures that the configuration file is always consistent with the dataset structure, making the training process more efficient and reducing the chances of misconfiguration.

```python
import yaml
import os

def create_data_yaml(path_to_classes_txt, path_to_data_yaml):

  # Read class.txt to get class names
  if not os.path.exists(path_to_classes_txt):
    print(f'classes.txt file not found! Please create a classes.txt labelmap and move it to {path_to_classes_txt}')
    return
  with open(path_to_classes_txt, 'r') as f:
    classes = []
    for line in f.readlines():
      if len(line.strip()) == 0: continue
      classes.append(line.strip())
  number_of_classes = len(classes)

  # Create data dictionary
  data = {
      'path': '/content/data',
      'train': 'train/images',
      'val': 'validation/images',
      'nc': number_of_classes,
      'names': classes
  }

  # Write data to YAML file
  with open(path_to_data_yaml, 'w') as f:
    yaml.dump(data, f, sort_keys=False)
  print(f'Created config file at {path_to_data_yaml}')
```

Figure 15

After preparing the dataset, the YOLOv11 model was trained using the Ultralytics YOLO training command in Google Colab.

The command specifies the dataset configuration file, selects the pretrained YOLOv11s model (yolo11s.pt), and initiates the training process for 100 epochs with an image size of 640×640 pixels and 16 batch. During training, the model learns to detect and classify different waste categories by analyzing labeled images in the dataset. The process automatically optimizes model weights, computes performance metrics such as loss, precision, and recall, and generates checkpoints to track improvements. Training results, including the best-performing model, are saved in the designated runs directory for later evaluation and testing.

```
!yolo detect train data=/content/data.yaml model=yolo11s.pt epochs=100 imgsz=640 batch=16
```

Figure 16

4. System Manual

4.1 Installation Usage

4.4.1 Model

The object detection model used in this project is YOLOv11. It was chosen because it works well in real time and is very accurate. Google Colab was used to develop and train the model. It is a cloud-based environment that is good for deep learning experiments. The labelled dataset was uploaded as a zip file and then unzipped into the working directory. The dataset was set up in the YOLO format and split into three parts: training, validation, and testing. The training set had 70% of the data, the validation set had 20%, and the testing set had 10%. After setting up the dataset paths and class definitions, the training process began with YOLOv11's training pipeline. After being trained, the model weights were tested with validation and test datasets to see how well they worked and how accurate they were at detecting things. Then they were used for real-time inference in the system.

https://colab.research.google.com/drive/1IkM1S0j85Q9aqL_PrhzBWVzlKJT7pW-P?usp=sharing

4.4.2 Data Collection

The dataset used in this project is a fully labeled dataset collected from both publicly available internet sources and images captured by our team. All images were carefully filtered to retain only high-quality and relevant samples, then precisely annotated according to predefined waste categories in YOLO format.

Figure 17

4.4.3 Hardware Device

Four main subsystems make up this project: camera, sensor, SBC, motors. The SBC will accept the input from all hardware subsystems and execute the YOLO model and handle all software functions, including image processing, YOLO model inference, handle incoming sensor data, Control the entire system. For this reason, the selected SBC is the NVIDIA Jetson Orin Nano due to its GPU-accelerated architecture and suitability for edge AI applications (IoT).

The camera is mounted on a plastic rod to support structure and connected directly to the SBC (Single Board Computer), so it is in a stable overhead position relative to the waste tray. By providing top-down coverage of all the objects, it also improves detection during the classification process.

The sensor system connects the wires to the SBC as well. It will be mounted on the side of the lid, detecting down toward the tray to detect the presence of objects on the tray. If an item is detected but is not classified as recyclable or hazardous, that item will automatically be moved to the general waste.

The custom developed future board includes a custom display structure to easily demonstrate tray assembly in conjunction with the lid so that the overall waste sorting procedure can be more easily visualized in an automated manner using AI based technology. The design of the lid is circular in nature, and the trays below the lid are separated into triangle sections as visual representations of the categories of waste; general waste, recyclable waste, and hazardous waste.

Each of the trays has a servo motor attached to the bottom of each tray, and every servo is wired directly to the SBC. When an object is found in the camera system, the YOLO AI model identifies and classifies the object and generates a control signal sent to the corresponding servo based on the type of waste found. The control signal activates the servo and adjusts the position of the tray to direct the waste into the appropriate bin.

This fully integrated mechanical/hardware combination represents an intelligent waste sorting system featuring automated, accurate, and efficient waste management capabilities.

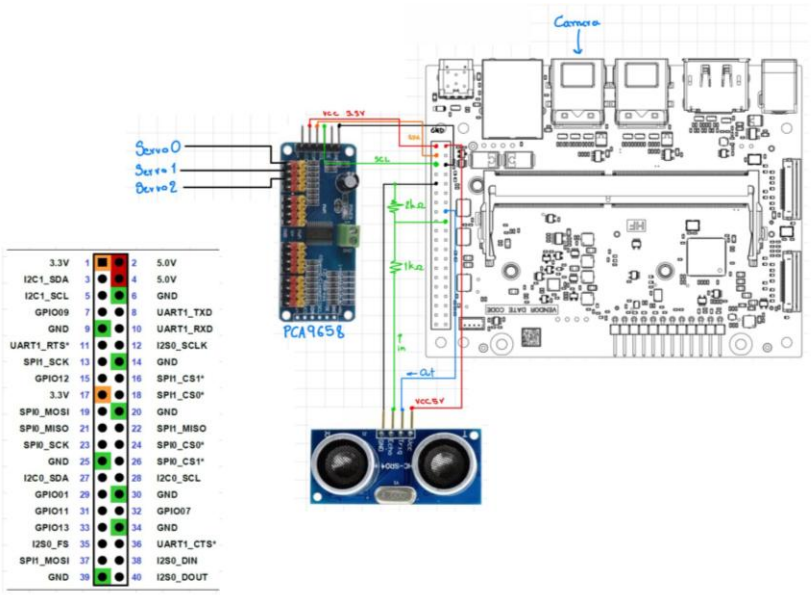
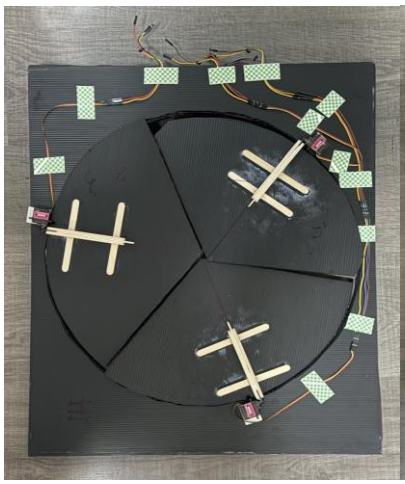


Figure 18



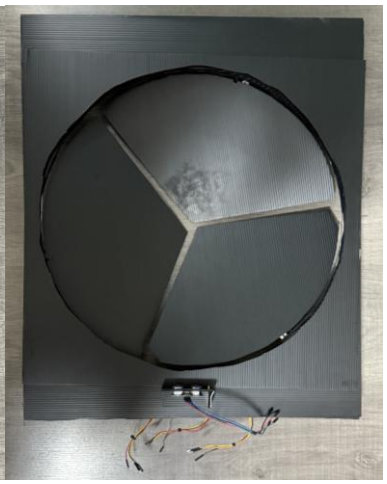


Figure 19 ***Figure 20***

Figure 21

Figure 22

4.2 Implementation Overview

The implementation of the project involves several key components working together to achieve the desired outcome. The main components are software implementation, hardware implementation, and system integration.

4.2.1 Software Implementation

The implementation of the software includes dataset, training the object detection models, and deploying them in real time. There are 10 labeled classes in our dataset: Battery, Can, Cutlery, Light Bulb, Phone, Water Bottle, Milk Carton, Plastic Cup. These classes are divided into two groups, one for recyclable items and one for hazardous waste items; anything that does not fall within the above 10 classes will be classified as general waste. The dataset was coded in the YOLO format for compatibility with the object detection framework. The models were trained using YOLO within the Google Colab environment. The labeled dataset was uploaded to Google Colab as a zip file and unzipped via the command *!unzip -q /content/data4.zip -d /content/custom_data* before starting the model training. The above process provides an efficient means of managing the dataset, training the models, and evaluating model performance prior to deployment for use in real-time object detection systems.

4.2.2 Hardware Implementation

At the current stage of the project, the system has been developed as a partially integrated hardware prototype that combines physical components with a computer-based AI processing system. A webcam is used to capture real-time images of waste items placed on a custom-designed tray, providing a clear top-down view for object detection. The captured frames are processed on a computer, where the trained YOLOv11 model performs real-time detection and classification of waste objects.

The SBC has been used as the main control unit in this system to take the input, preprocess the input, inference, and perform the overall control actions. Based on the predicted output and confidence of the YOLO model, the object is classified under certain type of garbage, and the result is converted into the control signal sent from SBC GPIO to servo motor to control it accordingly. The prediction of output triggers the corresponding servo by sending control signals either by manual intervention or simple interface for the control of servo to prove whether detection model is correct and mechanical sorting process can perform as intended.

In addition to the software component, a Single Board Computer (SBC) has been built and is connected to the servo motor, a sensor, and the physical sorting tray. This tray has several sections, classified into the type of trash such as recyclable, hazardous, general wastes, etc. The servo motor is attached to this tray to control its movement. Whenever it is activated, the servo moves to present how various waste is transported to respective sections/bins. There are three servos, namely servo 1 (Recycle), servo 2 (Dangerous), and servo 3 (General). If model predicts based on different conditions, servo 1 will activate

and trash belongs to Recycle, etc. In addition, a sensor is placed at the edge of the lid to detect the presence of an object to help the system classify the general waste. The sensor then sends a signal to the SBC to activate the corresponding servo motor for the general waste section.

4.2.3 System Integration

The system integration combines both software and hardware components into a unified pipeline that enables real-time intelligent waste sorting. The trained YOLO model, developed and validated in the Google Colab environment, is deployed onto the NVIDIA Jetson Nano, which acts as the central controller of the system.

The camera system continuously captures real-time images of the waste items being placed on the sorting tray and sends them to the SBC for preprocessing and object detection via the YOLO model. The camera system captures images of the items placed on the sorting tray in real-time throughout the operation of the camera system and sends them directly to the SBC. The camera system captures images of the items placed on the sorting tray in real-time during the operation of the camera system and sends the images directly to the SBC for preprocessing and object detection using the deployed YOLO model for prediction and confidence scoring.

Upon completion of object classification, the SBC converts the prediction results from the YOLO model into appropriate control signals via its GPIO interface to the corresponding servo motor associated with the sorting tray, thus driving the associated servo. Each of the three servos corresponds to a specific category of waste: Servo 1 controls recyclable waste, Servo 2 controls hazardous waste, and Servo 3 controls general waste. The servo will be set with a 2-second delay. The tray will open after the object is detected for 2 seconds, and once the object stops being detected, it will take 2 seconds to close the tray.

The sensor will be placed at the edge of the lid to detect objects. If it detects for more than 2 seconds, the general tray will open because it is not detected in the recycle or dangerous categories. Therefore, it will send a signal to the SBC to activate servo 2 (general), and the general tray will operate.

The integration of these components guarantees an interaction between perception, decision making, and actuation that is seamless. When the system detects an object, it activates the appropriate servo and changes the position of the tray to direct the waste to the appropriate compartment. The ability for this system to have closed-loop operation enables real-time operation and demonstrates the effectiveness of using AI-based object detection combined with embedded control hardware.

Ultimately, the prototype of this fully integrated system represents a working model of an automated waste sorting solution because it demonstrates the interdependencies between computer vision, embedded processing, and mechanical actuation.

```python
from adafruit_servokit import ServoKit
import time

# PCA9685 setup (16 channels)
kit = ServoKit(channels=16)

# Optional: better pulse range for MG90S servos
for i in range(3):
    kit.servo[i].set_pulse_width_range(500, 2500)

print("Testing 3 servos...")

while True:
    # -------------------------
    # Servo 1 (Channel 0)
    # -------------------------
    print("Servo 1 -> 0°")
    kit.servo[0].angle = 0
    time.sleep(2)

    print("Servo 1 -> 90°")
    kit.servo[0].angle = 90
    time.sleep(2)

    print("Servo 1 -> 180°")
    kit.servo[0].angle = 180
    time.sleep(2)

    # -------------------------
    # Servo 2 (Channel 1)
    # -------------------------
    print("Servo 2 -> 0°")
    kit.servo[1].angle = 0
    time.sleep(2)

    print("Servo 2 -> 90°")
    kit.servo[1].angle = 90
    time.sleep(2)

    print("Servo 2 -> 180°")
    kit.servo[1].angle = 180
    time.sleep(2)

    # -------------------------
    # Servo 3 (Channel 2)
    # -------------------------
    print("Servo 3 -> 0°")
    kit.servo[2].angle = 0
    time.sleep(2)

    print("Servo 3 -> 90°")
    kit.servo[2].angle = 90
    time.sleep(2)

    print("Servo 3 -> 180°")
    kit.servo[2].angle = 180
    time.sleep(2)
```

Figure 23

```python
import Jetson.GPIO as GPIO
import time

TRIG = 16   # Example: SPI1_CS1*
ECHO = 18   # Example: SPI1_CS0*

GPIO.setmode(GPIO.BOARD)

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, GPIO.LOW)
time.sleep(2)  # sensor settle time


def measure_distance():
    GPIO.output(TRIG, GPIO.HIGH)
    time.sleep(0.00001)
    GPIO.output(TRIG, GPIO.LOW)

    pulse_start = time.time()
    timeout = pulse_start

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()
        if pulse_start - timeout > 0.05:
            print("Timeout waiting for ECHO HIGH")
            return None

    pulse_end = time.time()
    timeout = pulse_end

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()
        if pulse_end - timeout > 0.05:
            print("Timeout waiting for ECHO LOW")
            return None

    pulse_duration = pulse_end - pulse_start

    distance = pulse_duration * 17150
    distance = round(distance, 2)

    return distance


try:
    print("Sensor test started")

    while True:
        dist = measure_distance()

        if dist is not None:
            print(f"Distance: {dist} cm")
        else:
            print("Measurement failed")

        time.sleep(1)

except KeyboardInterrupt:
    print("\nProgram stopped by user")

finally:
    GPIO.cleanup()
```

Figure 24

4.3 Test Result

Trained YOLOv11s for 100 epochs at 640 image resolution and tested its object detection capability with validation set. Overall, the result looks reasonable with a good balance between precision and recall. As we can see from the evaluation curves, F1-score gets peak of approximately 0.91 at confidence score of 0.709, a good balance was achieved. Also, high-confidence detections are very accurate; a perfect value of 1.00 was obtained for precision with confidence score of 0.995. The mAP@0.5 score of 0.927, the detection is a strong one.

From the evaluation per class, the classes mobile, can get very high detection scores 0.990, 0.982 respectively. Class cutlery (0.933), plastic cup (0.929) are also very good detection scores, whilst battery (0.895), lightbulb (0.901), milk carton (0.878), plastic bottle (0.911) are in a moderate range of high score detection.

It appears the detection performance varied slightly per class, this can be accounted to the difference of objects' sizes, similarity among classes and class distribution in training set. Thus, the score is a bit lower for objects which are very small and few in the dataset.

The hardware components were also tested as shown above for system integration. It was tested by having objects placed on the tray and the system was programmed to measure the distance to identify the object present using a distance measuring script and it successfully identified objects. Thus, it was verified that the sensor was able to read stable results and correctly identify objects over the course of 5s.

It was verified by testing servos using a script where each servo was successfully actuated using SBC to 0,90,180 degrees of movement to represent the 3 types of wastes, thus verified it can correctly control the movement.

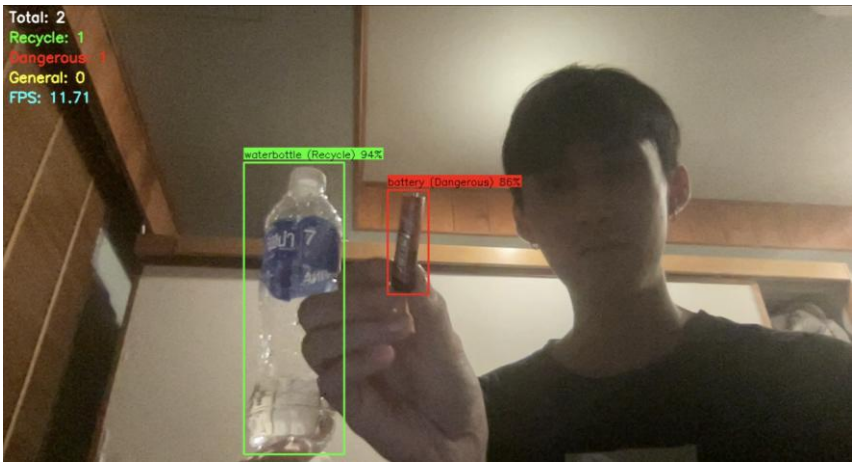


Figure 25

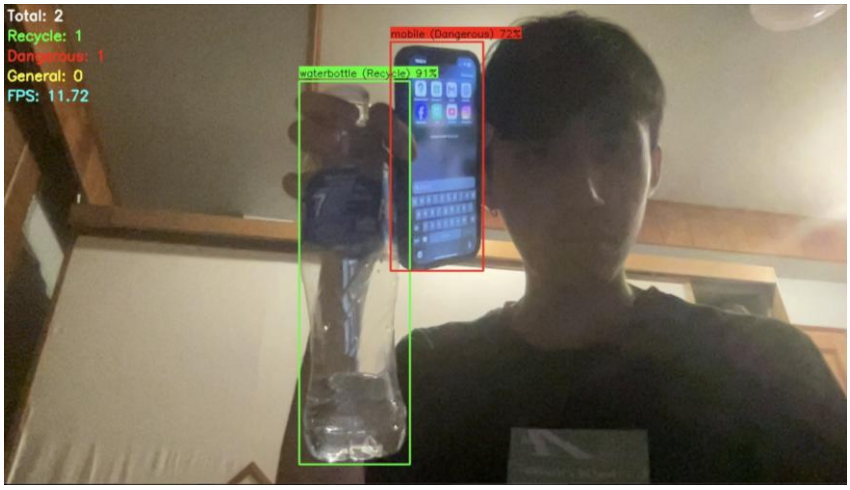


Figure 26

Conclusions

This project on waste detection model based on YOLO shows potential for automated waste sorting systems. The system can successfully detect multiple waste categories which show the whole system structure; training pipeline and model are well organized. The upgrade from MobileNetV2 to YOLO dramatically improves real-time performance of the system that is essential for real-world implementation of smart bins.

This system has significant improvement in dataset performance and model performance in this project. Besides improving the accuracy of labeling and image quality and class balancing, the team has also been able to design and manage custom dataset. Image under realistic scenarios is captured, labeled accurately, and data collection process is also better prepared to be compared to camera stream in real-time, hence achieving strong detection performance with about 0.91 F1-score and 0.927 mAP@0.5.

Besides software and dataset work, the system has successfully evolved hardware implementation and integration. The system includes SBC, camera module, sensor, and servo mechanism as waste sorting device. The YOLO model is run on SBC for real time inference and can send instructions to control sorting mechanisms. Based on the prediction from the system, appropriate servo motor will be activated and direct waste into the specified category. Sensor is integrated into the system to avoid wrongly categorized waste and deal with general waste.

Overall, the project completed a working prototype of a smart waste sorting system by using computer vision and embedded systems. The system achieves the right balance between accuracy, speed, and automation, therefore making the system applicable for real world implementation.

References

Introduction to Machine Learning with Label Studio. https://labelstud.io/blog/introduction-to-machine-learning-with-label-studio/

Suman Kunwar, Garbage Dataset: A Comprehensive Image Dataset for Garbage Classification and Recycling, Kaggle. https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2

Siddharth Sah, Plastic Bottles in the wild Image Dataset: 8000 ANNOTATED images of plastic bottles in the wild, Kaggle.
https://www.kaggle.com/datasets/siddharthkumarsah/plastic-bottles-image-dataset

Moezabid, Bottles and Cans Images, Kaggle.
https://www.kaggle.com/datasets/moezabid/bottles-and-cans

DataCluster Labs, Mobile Phone Dataset | Smartphone & Feature Phone, Kaggle.
https://www.kaggle.com/datasets/dataclusterlabs/mobile-phone-image-dataset?resource=download

Amir Hamza haq, Mobile Images Dataset, Kaggle.
https://www.kaggle.com/datasets/amirhamzahaq/mobile-images-dataset/data?select=images

YOGESH, Biomedical Waste Classification Images Dataset, Kaggle.
https://www.kaggle.com/datasets/mario78/biomedical-waste-classification-images-dataset

Bhushan Kinge, dry and wet waste Computer Vision Dataset, Roboflow
https://universe.roboflow.com/bhushan-kinge/dry-and-wet-waste/dataset/1